TP 11 : Jouer à l'Othello

Ce TP est à réaliser en `OCaml`.

Règles du jeu

On poursuit ici le travail de la semaine dernière. Vous avez à votre disposition les fichiers `TP10_A.ml`, `TP10_B.ml` et `TP10_C.ml` complétés qui implémentent les fonctions de la fois précédente. L'objectif est donc ici d'implémenter l'algorithme Min-Max dans un fichier `TP10_D.ml`

Rappels de type

Une case peut avoir trois états différents suivant sa couleur et son occupation. On propose donc d'utiliser le type suivant :

```
type case = Blanc | Noir | Vide
```

Une configuration du jeu est entièrement décrite comme l'état de l'ensemble des cases et le joueur dont c'est le tour. On utilisera donc le type suivant :

```
type configuration =
{
    plateau : case array array;
    joueur : case;
    nombre_passe : int
}
```

L'attribut `nombre_passe` compte le nombre de tours qui ont été passés depuis le dernier jeton posé. Ainsi, lorsqu'il atteint 2, la partie se termine puisqu'aucun joueur ne peut jouer.

Enfin, on représente un coup par la position du jeton placé :

```
type coup = int*int
```

Min-Max

Question 1 : Écrire une fonction `heuristique : configuration -> int` qui, étant donnée une configuration, renvoie la différence entre le nombre de cases blanches et le nombre de cases noires.

On commence tout d'abord par construire l'arbre des configurations de profondeur p. Pour cela, on utilise le type suivant :

```
type arbre_config = Noeud of configuration * arbre_config list
```

La liste représente l'ensemble des enfants tandis que la configuration représente l'état du jeu en ce nœud. Pour les feuilles, la liste sera donc vide.

Question 2 : Écrire une fonction `generer : configuration -> int -> arbre_config` qui, étant donné une configuration et une profondeur, renvoie l'arbre enracine en cette configuration.

On supposera dans la suite du sujet que l'on jouera toujours le joueur blanc.

Question 3 : Écrire une fonction `valeur : arbre_config -> (configuration -> int)-> int` qui, étant donné un arbre et une heuristique, calcule la valeur de la racine d'après l'algorithme Min-Max.

Question 4 : Écrire une fonction `trouver_coup : configuration -> (configuration -> int)-> int -> coup` qui, étant donné une configuration, une heuristique et une profondeur d'étude, renvoie le coup optimal d'après l'algorithme Min-Max.

Élagage

Question 5 : Pourquoi ne peut-on pas construire l'arbre si l'on veut effectivement appliquer l'élagage alpha-beta ?

Question 6 : Écrire une fonction `trouver_coup_rapide : configuration -> (configuration -> int)-> int -> coup` qui met en œuvre l'élagage alpha-beta pour calculer le coup optimal.